

A Extended Related Work

A.1 Robust PCA

Robust Principle Component Analysis (robust PCA) is the problem of finding a composition of a matrix M into a sum of sparse and low-rank components: $M = S + L$. It is a modification of PCA to accommodate corrupted observations (aka, noise). The sparse part covers the noise, while the low-rank part recovers the principle components. The most popular method to solve the problem is convex relaxation [8], where one minimizes the error $\|M - S - L\|_F^2$ subject to ℓ_1 constraint on $\|S\|_1$ and nuclear norm constraint on $\|L\|_*$, in order to promote the sparsity of S and the low-rankness of L . This convex problem can be solved with a variety of methods, such as interior point methods or the method of Augmented Lagrange Multipliers.

In our context, to find a sparse + low-rank decomposition of the attention matrix, one can also heuristically “peel off” the sparse part by finding the large entries of the attention matrix, then find a low-rank decomposition of the remainder. To avoid materializing the full attention matrix, one can use LSH to find potential locations of large entries, and use matrix completion [52] to find a low-rank decomposition. Gradient descent can find global optimum for this matrix completion problem [23]. However, it still requires too many iterations to be used in each training step.

A.2 Efficient Transformers

Sparse, Low-rank Approx.: Transformer-based model such as BERT [38] has achieved unprecedented performance in natural language processing. Recently, Vision Transformers [28, 69] has also achieved comparable performance to the traditional convolutional neural network in computer vision tasks [66]. However, the quadratic computation of the attention layers constrains the scalability of Transformers. There are many existing directions to overcome this bottleneck, including attention matrix approximation such as Reformer [36], Performer [17], leveraging a side memory module that can access multiple tokens at once [38, 39, 56] such as Longformer [5] and BigBird [70], segment-based recurrence such as Transformer-XL [19] and Compressive Transformer [49]. Please refer to a recent survey [58] for more details. In this paper, we mainly explore within the scope of approximating dense or full attention matrices.

Existing combination of Sparse and Low-rank Attention: Our focus on the classical and well-defined problem of matrix approximation, as opposed to simply designing an efficient model that performs well on downstream tasks (e.g., Longformer, Luna, Long-short transformer, etc.) affords us several advantages: (i) Easier understanding and theoretical analysis (Section 3, 4). We see that Scatterbrain yields an unbiased estimate of the attention matrix, and we can also understand how its variance changes. (ii) Clear-cut evaluation based on approximation error, as well as the ability to directly replace a full attention layer with Scatterbrain attention without re-training (Section 5). This setting is increasingly important as transformer models are getting larger and training them from scratch has become prohibitively costly. Other methods such as Luna and Long-short transformer are not backward compatible with pre-trained models.

Here we compare Scatterbrain with other work mentioned by the reviewer, showing how most of them are special cases of Scatterbrain. We will also add this discussion in the updated version of the manuscript.

- Longformer [5]: a special case of Scatterbrain where the sparse component is local attention, and the low-rank component is the global tokens. Global tokens can be considered a restricted form of low-rank approximation.
- BigBird [70]: a special case of Scatterbrain where the sparse component is local + random sparse attention, and the low-rank component is the global tokens. The use of global tokens makes the model unsuited for autoregressive modeling. On the other hand, Scatterbrain’s generality allows it to use other kinds of low-rank attention (e.g., Performer), and thus Scatterbrain works on both the causal/autoregressive and the bidirectional/non-causal attention settings. BigBird’s motivation is also quite different from ours: they aim to design efficient attention such that the whole Transformer model is still a universal approximator and is Turing complete. Our goal is more concrete and easier to evaluate: we approximate the attention matrices, to get a small Frobenius error between the Scatterbrain attention and the full attention matrices.
- Luna [43] (concurrent work): they use a fixed-length extra sequence and two consecutive attention steps: the context sequence attends to the extra sequence, and then the query sequence attends to the extra sequence. This is similar in spirit to low-rank attention (Linformer) and global tokens, but it is not a

low-rank approximation due to the non-linearity between the two attention steps. It is not clear to us that it combines different kinds of attention.

- Long-short transformer[71] (concurrent work): a special case of Scatterbrain where the sparse component is local attention and the low-rank component is Linformer.

A.3 Locality Sensitive Hashing for Efficient Neural Network Training

Locality Sensitive Hashing (LSH) has been well-studied in approximate nearest-neighbor search [2, 11, 27, 30, 34, 54]. Since the brute-force approach for similarity search is computationally expensive, researchers have come up with various indexing structures to expedite the search process. Usually this comes with trade-offs on the search quality. Based on these indexing structures, one can achieve sub-linear search time. LSH has been used in estimation problem as well [12, 13].

Recently, there has been several work taking advantage of LSH data structures for efficient neural network training. During training process, the weight matrices are slowly modified via gradients derived from objective functions. If we consider the weights as the search data and input as queries, we can view neural network training as a similarity search problem. For example, [14, 18, 41] proposes an algorithm which performs sparse forward and backward computations via maximum inner product search during training. It is based on the observation that the model is usually over-parameterized so the activation for a given input could be sparse and LSH is used to find or impose the sparse structure. Similarly, LSH based algorithms have also been used in Transformers [14, 15], where LSH is used to capture the sparse structure of the attention matrices. They can largely reduce the memory bottleneck of self-attention modules especially over long sequences in Transformer. Though [15] has done some exploration to improve LSH accuracy-efficiency trade-offs through learnable LSH, most of the above works have limited understanding on when and where LSH can perform well.

A.4 Structured Matrices for Efficient Machine Learning Models

Sparse + low-rank is an example of a class of structured matrices: those with asymptotically fast matrix-vector multiplication algorithm ($o(n^2)$ time complexity) and few parameters ($o(n^2)$ space complexity). Common examples include sparse, low-rank matrices, and matrices based on fast transforms (e.g., Fourier transform, circulant, Toeplitz, Legendre transform, Chebyshev transform, and more generally orthogonal polynomial transforms). These classes of matrices, and their generalization, have been used in machine learning to replace dense matrices in fully connected, convolutional, and recurrent layers [32, 55, 61]. De Sa et al. [24] shows that any structured matrix can be written as product of sparse matrices, and products of sparse matrices even with fixed sparsity pattern have been shown to be effective at parameterizing compressed models [1, 20, 21].

In our setting, it remains difficult to approximate the attention matrix with these more general classes of structured matrices. This is because many of them are fixed (e.g., Fourier transform, orthogonal polynomial transforms), and there lacks efficient algorithms to find the closest structured matrix to a given attention matrix.

B Motivating Observations: Low-rank and Sparse Structures of Attention Matrices

We aim to build a deeper understanding of sparse and low-rank structures in real attention matrices: where each of them excel, and the potential for their combination. Specifically, we

- show that sparse and low-rank approximation errors are negatively correlated (through statistical tests),
- characterize regimes where each of sparse and low-rank approximation are well-suited, as dictated by the entropy of the softmax attention distribution, and
- demonstrate that sparse + low-rank has the potential to achieve better approximation than either.

B.1 Setup

Denote M as the attention matrix (after softmax) and $\mathcal{H}$ as entropy. We measure approximation error by the Frobenius norm or the original matrix and the approximation (sparse or low-rank). All the observed attention matrices in this section are from (1) a 4-layer vanilla Transformer trained from scratch on char-level IMDb reviews classification [57] (2) a 16-layer vanilla Transformer trained from scratch on WikiText103 [45] (3) a 1-layer (attention) pre-trained BigGAN on ImageNet [25]. To collect attention matrices for IMDb and WikiText103, we first save checkpoint of the models in every epoch; then evaluate 100 samples from validate data for each checkpoint and collect attention matrices from each layer each head. Note we take the median of the stats (error) for those 100 samples if it is difficult to visualize. To collect attention matrices for BigGAN, we generate 100 samples and collect the attention on the fly.

B.2 Observation 1: Sparse and low-rank approximation errors are negatively correlated

Table 3: The Spearman’s rank, Pearson and Kendall’s Tau correlation coefficients between Sparse and Low-rank approx. error on IMDb, WikiText-103, and BigGAN-ImageNet. P-values of < 0.05 indicate statistical significance. The two errors are negatively correlated.

	IMDb		WikiText103		BigGAN-ImageNet	
	Coef	p-value	Coef	p-value	Coef	p-value
Spearman’s rank	-0.89	$< .00001$	-0.63	$< .00001$	-0.21	$< .00001$
Pearson	-0.78	$< .00001$	-0.61	$< .00001$	-0.31	$< .00001$
Kendall’s Tau	-0.74	$< .00001$	-0.51	$< .00001$	-0.18	$< .00001$

We fixed the number of parameters, K , allowed for each attention matrix approximation and collect the errors from ideal sparse and low-rank approximations: top- K entries for each row of the matrix for sparse and top- K eigenvalues for low-rank. Then we run three standard statistical correlation tests [4, 59], Spearman, Pearson and Kendall’s Tau on sparse and low-rank approximation error for all the matrices. We can see from Table 3 that errors are significantly negatively correlated (p-value < 0.05). Further more, the left three plots on Figure 7 visualizes the correlation between the two errors on three datasets.

This negative correlation suggests that there is some property of the softmax attention distribution which determines when sparse or low-rank excels. We validate this claim in the next observation.

B.3 Observation 2: Sparse approximation error is lower when softmax entropy is low and low-rank approximation error is lower error when entropy is high

We visualize the sparse and low-rank approximation error against the entropy of attention matrices $\mathcal{H}(M)$ (applied to each row, then averaged) on the right plot in Figure 7. The attention matrices are $\in \mathbb{R}^{1024 \times 1024}$ (padded) so the x-axis has range from $[0, \ln(1024)]$. For high-entropy distributions (more diffused) low-rank

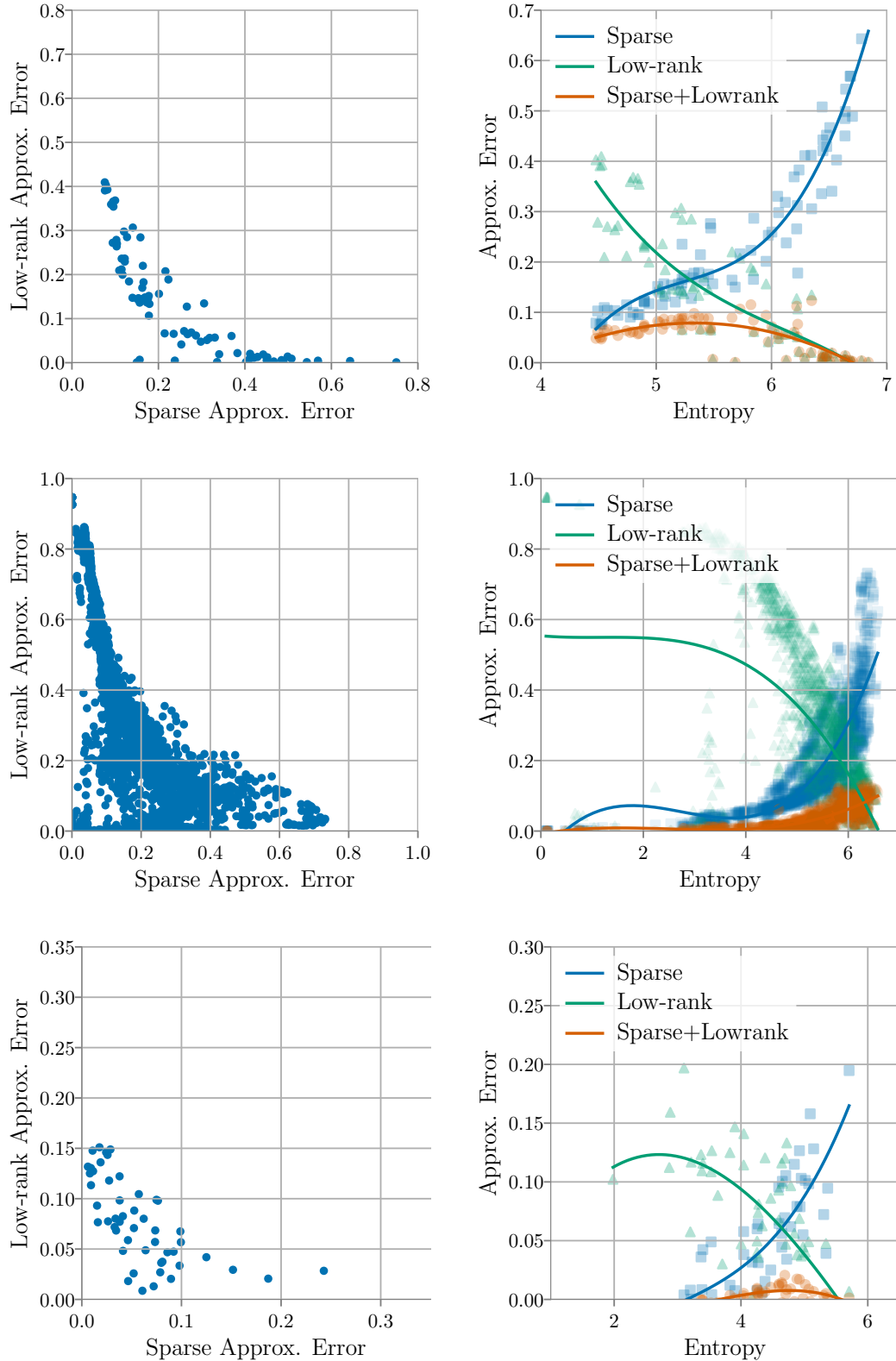


Figure 7: Characterization of the relationship between the softmax distribution of each attention matrix row and approximation error of sparse, low-rank and sparse+low-rank. The top, middle and bottom plots are for IMDB, WikiText103 and BigGAN-ImageNet respectively. Left: The approximation error of sparse and low-rank are negatively correlated. Sparse performs well when low-rank does not, and vice versa. Right: Entropy of the softmax attention distribution (i.e., scale of logits) determines the regimes where sparse, low-rank, or sparse + low-rank perform well. Sparse + low-rank yields better approximation than sparse or low-rank alone, across the board.